Job Market Analytics Dashboard

Step 4 — Supabase Database Setup & Data Storage

Step 4 Overview

Status: ✓ Complete

Created a brand new dedicated Supabase project for the dashboard (separate from the portfolio), created the jobs table using SQL, granted permissions, installed the supabase Python library, and successfully inserted 60 live job records into the database — 20 Python developer, 20 data scientist, and 20 machine learning jobs.

4.1 Why a Separate Supabase Project?

The portfolio already uses its own Supabase project. We deliberately created a brand new separate project for the dashboard so that:

Reason	Detail
Independence	Pausing, deleting, or upgrading one project does not affect the other
Separate API keys	Each project has its own URL and keys — no sharing or confusion
Clean data separation	Portfolio projects table stays separate from job market data
Professional architecture	Two deployed apps, two databases — looks like real production work
Easier debugging	If dashboard breaks, portfolio still works and vice versa

4.2 New Supabase Project Details

Setting	Value
Project name	job-market-dashboard
Project ID	vlrdniwvjdytzdknokeq
Project URL	https://vlrdniwvjdytzdknokeq.supabase.co
Region	Southeast Asia (Singapore) — ap-southeast-1
Database	Primary Database — Nano compute
RLS	Disabled (public read dashboard — job data is not private)

4.3 jobs Table — Created via SQL Editor

SQL used to create the table:

```
CREATE TABLE jobs (
id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
title text,
company text,
location text,
salary_min float8,
salary_max float8,
skills text[],
category text,
source_url text,
posted_date text,
description text,
scraped_at timestamptz DEFAULT now()
);
```

Column explanations:

Column	Type	Why this type
id	uuid	Auto-generated unique ID — safer than integers for public APIs
title	text	Variable length job title
company	text	Company name
location	text	City/region string
salary_min	float8	Decimal salary — nullable (many jobs don't list salary)
salary_max	float8	Decimal salary — nullable
skills	text[]	Array of strings — Postgres native array type
category	text	Job category from Adzuna
source_url	text	Used for duplicate detection
posted_date	text	ISO timestamp string from API
description	text	First 500 chars of job description
scraped_at	timestamptz	Auto-set to current time on insert

4.4 Permission Fix

New Supabase projects require explicit permission grants for the API roles to access tables. Without this, all queries return "permission denied".

ERROR encountered:

```
postgrest.exceptions.APIError: permission denied for table jobs (code 42501)
```

Fix — run this in SQL Editor:

```
GRANT ALL ON public.jobs TO anon;
GRANT ALL ON public.jobs TO authenticated;
GRANT ALL ON public.jobs TO service_role;
```

Result: ✓ Permissions granted. All subsequent queries work correctly.

4.5 Install Supabase Library

```
pip install supabase
pip freeze > requirements.txt
```

4.6 scraper/store_jobs.py — Full Code

```
import os
from supabase import create_client
from dotenv import load_dotenv
from fetch_jobs import fetch_jobs
from clean_data import clean_jobs
load_dotenv()
supabase = create_client(
os.getenv("SUPABASE_URL"),
os.getenv("SUPABASE_SERVICE_KEY")
)
def store_jobs(what="python developer", where="london"):
print(f"Fetching jobs for '{what}' in '{where}'...")
raw = fetch_jobs(what=what, where=where)
df = clean_jobs(raw)
inserted = 0
skipped = 0
for _, row in df.iterrows():
# Check if job already exists by source_url (duplicate prevention)
existing = supabase.table("jobs").select("id").eq("source_url", row["source_url"]).execute()
```

```
if existing.data:
skipped += 1
continue
supabase.table("jobs").insert({
"title": row["title"],
"company": row["company"],
"location": row["location"],
"salary_min": row["salary_min"] if row["salary_min"] else None,
"salary_max": row["salary_max"] if row["salary_max"] else None,
"skills": row["skills"],
"category": row["category"],
"source_url": row["source_url"],
"posted_date": row["posted_date"],
"description": row["description"],
}).execute()
inserted += 1
print(f"Done — {inserted} inserted, {skipped} skipped (duplicates)")
if __name__ == "__main__":
store_jobs("python developer", "london")
store_jobs("data scientist", "london")
store_jobs("machine learning", "london")
```

4.7 How Duplicate Prevention Works

Before inserting any job, we check if a record with the same source_url already exists. source_url is unique per job posting — two different jobs will never share the same URL. If it exists, we skip it. This means running the scraper multiple times will not create duplicate records.

4.8 Actual Output

Search Query	Location	Fetched	Inserted	Skipped
python developer	london	20	20	0
data scientist	london	20	20	0
machine learning	london	20	20	0
TOTAL		60	60	0

Result: ✓ 60 jobs stored in Supabase jobs table.

4.9 .env File (Current State)

Your .env now contains all required keys for this project:

```
ADZUNA_APP_ID=87928b97
ADZUNA_API_KEY=5f8132d6176bc5b4253722cca6c910cb
SUPABASE_URL=https://vlrdniwvjdytzdknokeq.supabase.co
SUPABASE_KEY=sb_publishable_... (full publishable key)
SUPABASE_SERVICE_KEY=sb_secret_... (full secret key)
```

Note: SUPABASE_KEY (publishable) is used for read operations. SUPABASE_SERVICE_KEY (secret) is used for write operations like inserting jobs. Never commit these to GitHub — .env is in .gitignore.

Build Progress

Step	Description	Status
1	Project setup — venv, packages, file structure, local Flask server	✓ Done
2	Adzuna API — register, get keys, write fetch_jobs.py, pull live data	✓ Done
3	Data cleaning — Pandas, normalize fields, extract skill tags	✓ Done

4	Supabase — create jobs table, store first batch of 60 jobs	✓ Done
5	Dashboard route — query Supabase, pass data to templates	Next →
6	Plotly charts — skills, location, salary charts	
7	Jobs listing page — /jobs route with filters	
8	UI polish — dark CSS theme	
9	GitHub + Railway deployment	
10	Add to portfolio via /admin	
11	(Stretch) ML salary prediction model	